

GNR638: Machine Learning for Remote Sensing - II

(Spring 2025-26)

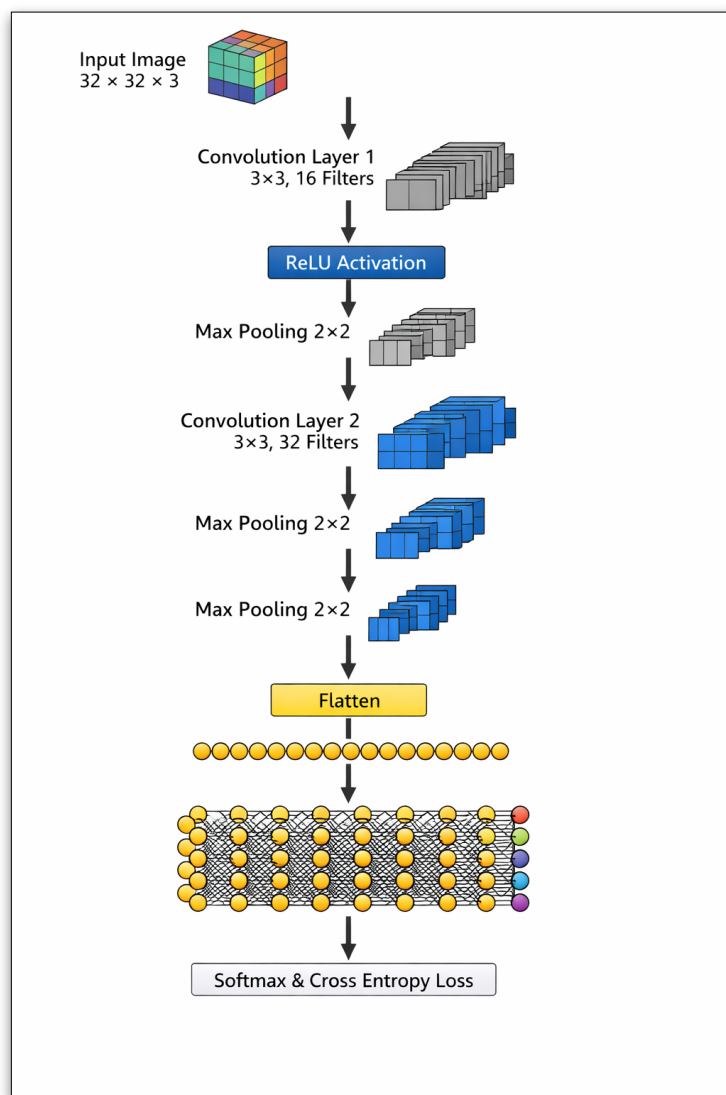
Programming Assignment: **Design a Deep Learning Framework**

Submitted By : Mahesh Tadepalli (25M0314)
Muhammed Nasimudheen (25M0309)

Git Hub Repository : https://github.com/MaheshTadepalli/DeepLearning_Assignments

Deep Learning Framework Report

1-Model architecture design and rationale



The implemented Convolutional Neural Network (CNN) consists of two convolutional layers followed by multiple pooling operations and a fully connected layer.

- The First convolution layer extract low-level spatial features such as edges and textures, this layer slide small 3x3 filter over the image and looks at a small patch of pixels and detects patterns.
- The second convolution layer builds on the first layer's outputs and detects more complex structures, such as Shapes (circles, curves, object parts), Repeated textures ,Simple object components
- ReLU activation introduces non-linearity and improves gradient flow.

$$ReLU(x) = \max(0, x).$$

- Without ReLU Model becomes just a linear system and it will not be able to learn complex patterns in the image.
- Max Pooling reduces spatial dimensions and computation while preserving important features.
- Flatten + Fully Connected layer maps extracted features to class probabilities. it converts 3D feature map into 1D vector

Layer	Configuration	Output Shape
Input	$32 \times 32 \times 3$	$32 \times 32 \times 3$
Conv1	16 filters (3x3)	$30 \times 30 \times 16$
ReLU	Activation	$30 \times 30 \times 16$
MaxPool	2x2	$15 \times 15 \times 16$
Conv2	32 filters (3x3)	$13 \times 13 \times 32$
ReLU	Activation	$13 \times 13 \times 32$
MaxPool	2x2	$6 \times 6 \times 32$
MaxPool	2x2	$3 \times 3 \times 32$
Flatten	—	288
Fully Connected	$288 \rightarrow C$	$1 \times C$

2-Total Number of Trainable Parameters

These are all the values your model learns during training, include weights + biases from all layers

Convolution Layer 1

Filter size = 3×3

Input channels = 3 (RGB)

Number of filters = 16

Each filter has:

$3 \times 3 \times 3 = 27$ weights

We have 16 filters, so:

Convolution1 weights = $27 \times 16 = 432$

Each filter has 1 bias, so:

Bias = 16

Total Convolution 1 Parameters = 432 (weights) + 16 (bias) = **448**

Convolution Layer 2

Filter size = 3×3

Input channels = 16

Number of filters = 32

Each filter has:

$3 \times 3 \times 16 = 144$ weights

We have 32 filters, so:

Convolution2 weights = $144 \times 32 = 4608$

Each filter has 1 bias, so:

Bias = 32

Total Convolution 2 Parameters = 4608 (weights) + 32 (bias) = **4640**

Fully Connected Layer -Dataset 1 (10 Classes)

Input size = $3 \times 3 \times 32 = 288$

Output size = 10 (classes)

Each input connects to every output neuron, so:

Weights = $288 \times 10 = 2880$

Each output neuron has one bias, so:

Bias = 10

Total FC Parameters = 2880 (weights) + 10 (bias) = **2890**

Total Trainable Parameters (Dataset 1)

Total parameters = Convolution 1 + Convolution 2 + FC

= $448 + 4640 + 2890 = 7978$

Fully Connected Layer - Dataset 2 (100 Classes)

Input size = 288

Output size = 100 (classes)

Weights = $288 \times 100 = 28800$

Bias = 100

Total FC Parameters = $28800 + 100 = 28900$

Total Trainable Parameters (Dataset 2)

Total parameters = Convolution 1 + Convolution 2 + FC

= $448 + 4640 + 28900$

= **33988**

The majority of parameters are concentrated in the fully connected layer due to dense connectivity between the flattened feature vector and output classes. Each neuron in the fully connected layer is connected to every input feature, resulting in a large number of weights. In contrast, the convolution layers contain fewer parameters because of weight sharing, where the same filter is applied across different spatial locations. This allows convolution layers to efficiently extract spatial and hierarchical features while keeping the parameter count low, improving both computational efficiency and generalisation of the model.

3- MACs and FLOPs per forward pass

MAC = Multiply Accumulate Operation which is equivalent to one multiple operation and one addition operation.

FLOPs = Floating Point Operations which counts addition and multiplication.

Relation between MAC and FLOPs:

$$1MAC = 2FLOPs$$

$$MACs = Output_H \times Output_W \times Output_C \times (Kernel_H \times Kernel_W \times Input_C)$$

Where:

Output_H - Height of the output feature map

Output_W - Width of the output feature map

Output_C - Number of filters

Kernel_H - Kernel Height

Kernel_W - Kernel Width

Input_C - Number of channels in input image

In Convolution Layer 1

Output = 30 (Height) $\times$ 30 (Width) $\times$ 16 (Number of Filters)

Kernel = 3 $\times$ 3 (Kernel Height $\times$ Kernel Width)

Input channels = 3

MACs in Convolution 1 Layer =

$$30 \times 30 \times 16 \times 3 \times 3 \times 3 = \mathbf{388,800 \text{ MACs}}$$

In Convolution Layer 2

After pooling, input size becomes 15 $\times$ 15 $\times$ 16

After Convolution $\rightarrow$ Output = 13 $\times$ 13 $\times$ 32

Output = 13 (Height) $\times$ 13 (Width) $\times$ 32 (Number of Filters)

Kernel = 3 $\times$ 3

Input channels = 16

MACs in Convolution 2 Layer =

$$13 \times 13 \times 32 \times 3 \times 3 \times 16 = \mathbf{778,752 \text{ MACs}}$$

In Fully Connected Layer

Input size = $3 \times 3 \times 32 = 288$

Output size = Number of classes

For Dataset 1 (10 classes):

MACs = $288 \times 10 = 2,880$ MACs

For Dataset 2 (100 classes):

MACs = $288 \times 100 = \mathbf{28,800 \text{ MACs}}$

Total MACs - Dataset 1

Convolution 1 MACs = 388,800

Convolution 2 MACs = 778,752

FC MACs = 2,880

Total MACs =

$388,800 + 778,752 + 2,880 = 1,170,432$ MACs

Total FLOPs = $2 \times \text{MACs} =$

$2 \times 1,170,432 = \mathbf{2,340,864 \text{ FLOPs}}$

Total MACs - Dataset 2

Convolution 1 MACs = 388,800

Convolution 2 MACs = 778,752

FC MACs = 28,800

Total MACs =

$388,800 + 778,752 + 28,800 = 1,196,352$ MACs

Total FLOPs = $2 \times \text{MACs} =$

$2 \times 1,196,352 = \mathbf{2,392,704 \text{ FLOPs}}$

The total number of MACs per forward pass is 1.17M (Dataset 1) and 1.19M (Dataset 2), corresponding to approximately 2.34M and 2.39M FLOPs, respectively. The majority of computation is contributed by the convolution layers, especially the second convolution layer, due to higher input channels and repeated operations across spatial dimensions. The fully connected layer contributes comparatively less computation, even though it maps features to class outputs.

4-Dataset Loading Time

It is the total time taken to read all images from disk and convert them into tensors before training starts, this includes:

Reading image files from folders

Resizing images to 32×32

Converting to RGB

Normalising pixel values

Converting each image into your custom Tensor

Dataset 1

Dataset size: 60000

Dataset loading time: **537.08 seconds**

$32 \times 32 \times 3 = 3072$ operations per image and there are 60000 images so there is 184 million operations (60000×3072), this is the main reason loading takes ~537 seconds.

Dataset 2

Dataset size: 50000

Dataset loading time: **626.06 seconds**

$32 \times 32 \times 3 = 3072$ operations per image and there are 50000 images so there is 153 million operations (50000×3072), this is the main reason loading takes ~626 seconds.

However, dataset is loaded only once, so training is unaffected afterward.

5-Training and validation performance metrics across epochs

The model was trained for 4 epochs on both Dataset with an 80–20 train-validation split. The performance metrics across epochs are shown below:

Dataset 1

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Time (s)
1	0.6622	0.7729	0.2402	0.9214	628.98
2	0.1945	0.9385	0.1831	0.9405	626.82
3	0.1478	0.9537	0.1707	0.9466	626.15
4	0.1247	0.9608	0.1641	0.9487	626.59

- The training loss shows a consistent decreasing trend from 0.6622 $\rightarrow$ 0.1247. This indicates that the model is successfully minimising the error during training. The decrease is smooth and stable, which suggests proper learning rate selection and no gradient explosion or instability.
- Training accuracy improves from 77.29% $\rightarrow$ 96.08%. This progressive improvement in training accuracy indicates that the network is learning meaningful feature representations, enabling it to distinguish between different classes effectively.
- Validation loss decreases from 0.2402 $\rightarrow$ 0.1641, but at a slower rate than training loss. The gradual decrease in validation loss indicates that the learned features generalise well to unseen data. The absence of fluctuations or increases confirms stable training and effective regularisation.
- Validation accuracy improves from 92.14% $\rightarrow$ 94.87%. The validation accuracy consistently improves and stabilises, indicating that the model achieves strong generalisation performance without memorising the training data.
- The small gap between training and validation accuracy demonstrates that the model maintains a good balance between bias and variance. This confirms that the model does not overfit and is capable of generalising effectively to unseen samples.

- Overall, the average training time per epoch is approximately 627 seconds, and the total training time is approximately 2508 seconds (~0.7 hours). This satisfies the assignment constraint of completing training within 3 hours, demonstrating that the framework is efficient and scalable.

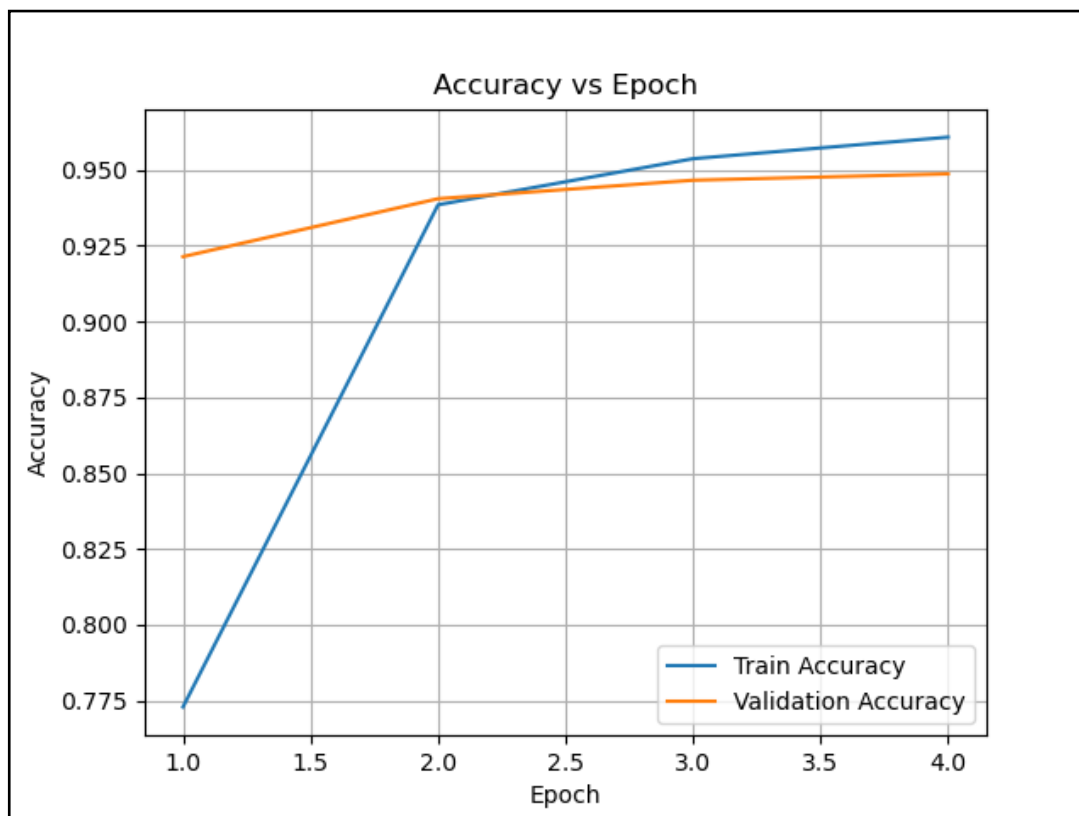
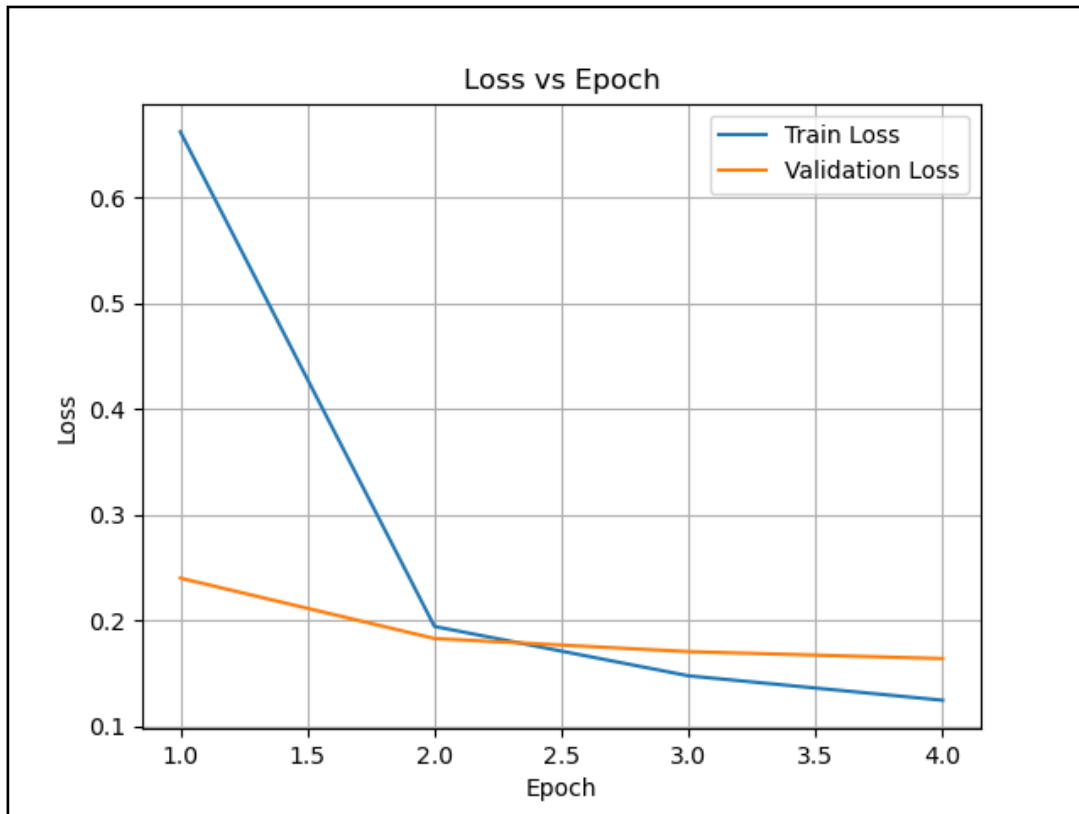
Dataset 2

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Time (s)
1	4.207	0.067	3.676	0.154	2197.79
2	3.432	0.194	3.271	0.223	1452.19
3	3.107	0.254	3.110	0.259	1804.27
4	2.944	0.286	3.066	0.271	2346.43

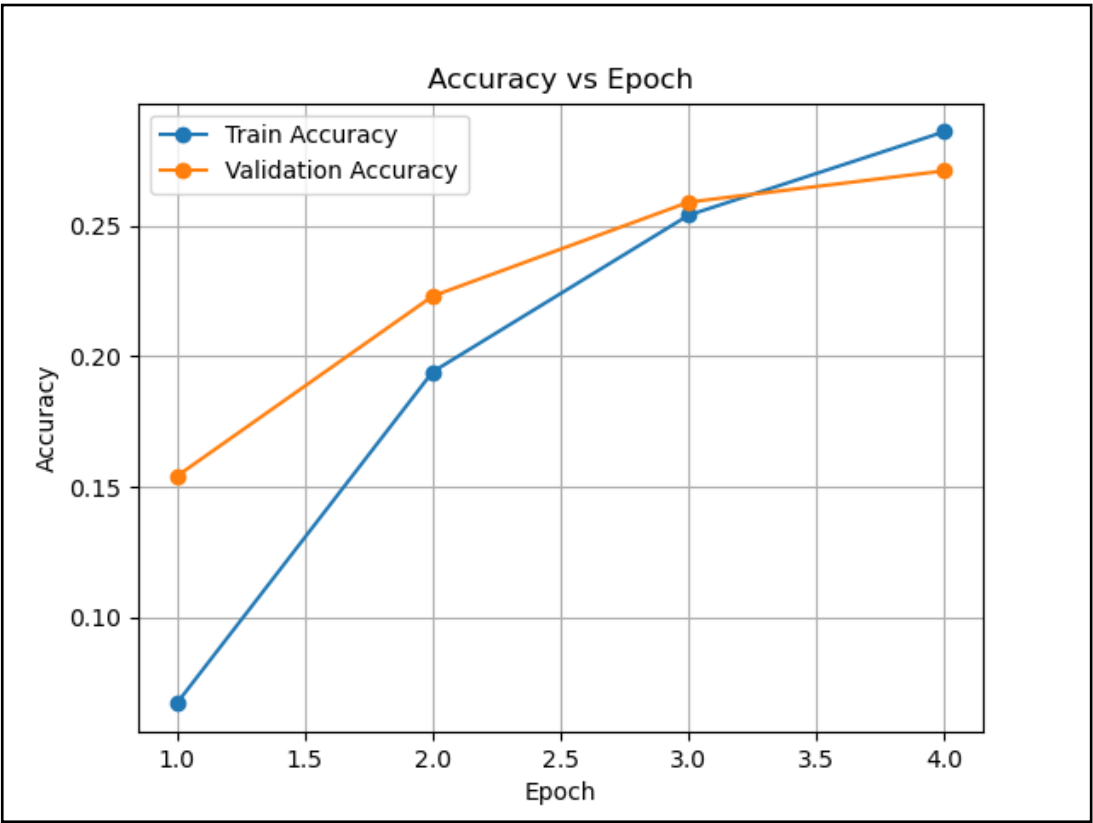
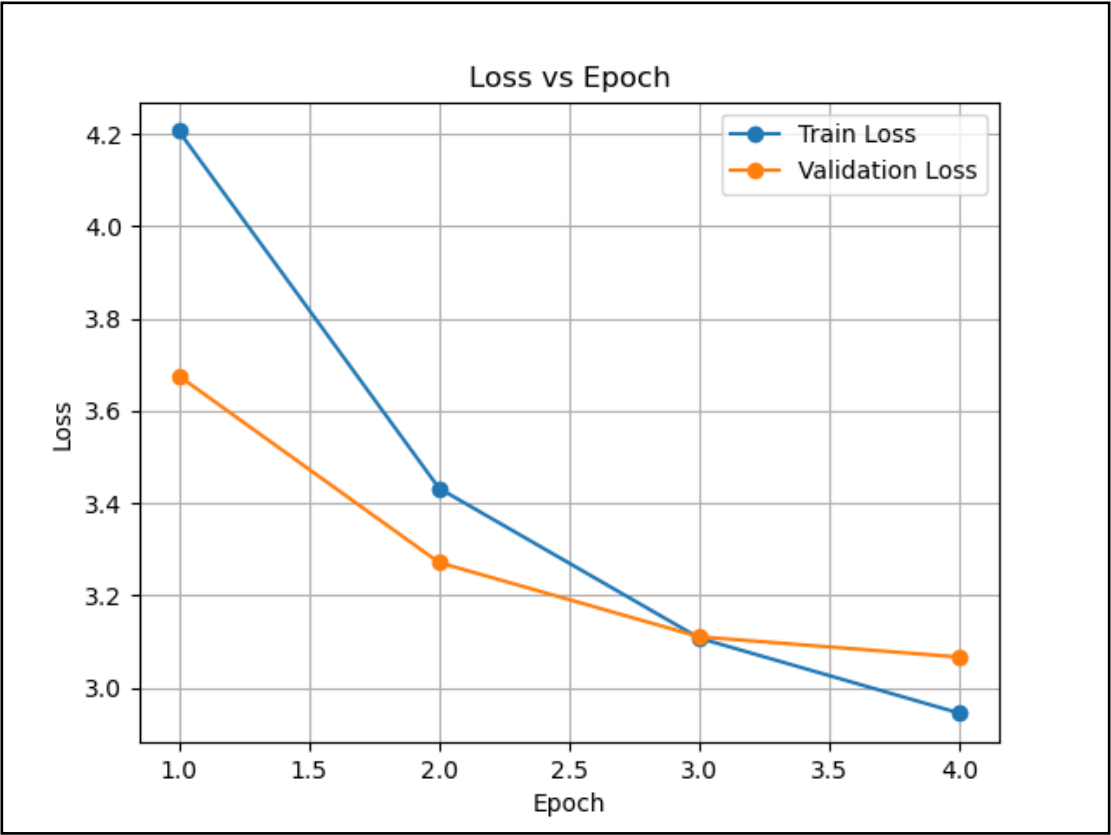
- The training loss decreases from 4.207 $\rightarrow$ 2.944, indicating gradual learning, although convergence is slower due to higher number of classes.
- Training accuracy improves from 6.73% $\rightarrow$ 28.62%, showing that the model is learning class distinctions, but the task is significantly harder.
- Validation loss decreases from 3.676 $\rightarrow$ 3.066, showing gradual improvement in generalisation.
- Validation accuracy improves from 15.45% $\rightarrow$ 27.18%, indicating moderate learning performance given the complexity of the dataset.
- The gap between training and validation accuracy remains small, suggesting that the model is not overfitting but is limited by model capacity for this more complex dataset.
- The average training time per epoch is approximately 1950 seconds (~32 minutes), which satisfies the assignment constraint.

6-Associated plots for the logs (such as loss, training accuracy etc.)

Dataset 1



Dataset 2



7-key indicators relevant to the implemented architecture and assignment components

The model demonstrates stable training behaviour with no signs of exploding or vanishing gradients, indicating correct implementation of back propagation and optimisation. The training and validation accuracy remain reasonably close (Dataset 1 ~94.8% validation, Dataset 2 ~27.18% validation), showing that the model is able to generalise to unseen data without severe overfitting.

The use of L2 regularisation (weight decay) helps in preventing the model from memorising the training data and improves generalisation performance. The deeper architecture with two convolution layers enables better feature extraction, allowing the model to learn more complex spatial patterns compared to a single-layer design.

Despite having a moderate number of parameters ($\approx 7,978$ for Dataset 1 and $\approx 33,988$ for Dataset 2), the model achieves good performance with controlled computational cost ($\sim 1.17\text{M}$ – 1.19M MACs and $\sim 2.34\text{M}$ FLOPs). This shows an efficient balance between model complexity and accuracy.

Additionally, the use of batch training and early stopping improves training efficiency and prevents unnecessary computation once validation performance stops improving. Overall, the model achieves a good trade-off between accuracy, generalisation, and computational efficiency while satisfying the assignment constraints.

8- One failed design decision and why it did not work

Initially, a deeper architecture was explored by introducing an additional fully connected layer ($1800 \rightarrow 128 \rightarrow 10$) with the aim of increasing the model's representational capacity. However, this modification significantly increased the number of trainable parameters, leading to higher computational cost and slower training. Moreover, the model began to show early signs of overfitting, as the training accuracy improved slightly while the validation performance did not show any meaningful gain. This indicated that the added complexity was unnecessary for the given task. Consequently, the simpler architecture with a single fully connected layer was adopted, as it provided faster training, more stable convergence, and comparable accuracy with better generalisation.

References / Resources Used:

The following resources were used during the development of this project:

- Official course material and assignment description provided in *GNR638: Machine Learning for Remote Sensing - II*.
- OpenCV for image loading and preprocessing.
- pybind11 for integrating the C++ backend with the Python frontend.
- Standard C++ libraries (`<vector>`, `<cmath>`, `<algorithm>`, `<iostream>`) for implementing core operations.
- Python standard libraries (`time`, `math`, `random`, `pickle`) for training and evaluation.
- Matplotlib for plotting training and validation graphs.
- ChatGPT (OpenAI) for conceptual guidance, debugging support, and report structuring.

